

**Universidad
Tres culturas**

**Materia:
Desarrollo para Móviles**

**Tema:
Documentación técnica de la App
MenuRestaurante3**

**Carrera:
Ingeniería en Sistemas Computacionales**

**Profesor:
Hernandez Crasborn Erendira de los
Angeles**

**Alumno:
Campos Sanchez Victor Abraham**

Índice

1. Introducción
2. Objetivo general
3. Objetivos específicos
4. Descripción general de la aplicación
5. Tecnologías utilizadas
6. Arquitectura de la solución
7. Diagrama de arquitectura MVC
8. Integración con Firebase
9. Modelo de datos en Cloud Firestore
10. Diagrama del modelo de datos en Firestore
11. Operaciones CRUD
12. Evidencias del funcionamiento
13. Pruebas realizadas
14. Instalación y uso
15. Conclusiones

1. Introducción

MenuRestaurante3 es una aplicación móvil desarrollada en Android Studio utilizando Kotlin para la gestión de productos, mesas y pedidos de un restaurante. Permite autenticar usuarios mediante Firebase Authentication, consultar el menú, seleccionar una mesa, agregar productos, calcular el total y administrar pedidos.

El proyecto utiliza una arquitectura MVC, complementada con clases DAO para separar el acceso a datos. Se emplea SQLite para el almacenamiento local y Cloud Firestore para almacenar los pedidos en la nube. También se implementan las operaciones CRUD y la sincronización de cambios relevantes con Firebase.

2. Objetivo general

Desarrollar una aplicación móvil para gestionar pedidos de un restaurante mediante Kotlin y Android Studio, utilizando arquitectura MVC, SQLite, Firebase Authentication y Cloud Firestore para administrar usuarios y pedidos.

3. Objetivos específicos

1. Implementar registro, inicio y cierre de sesión mediante correo electrónico y contraseña.
2. Desarrollar una interfaz para consultar el menú, seleccionar mesas y agregar productos.
3. Permitir crear pedidos, calcular su total y administrar sus productos.
4. Utilizar SQLite para el almacenamiento local de la información.
5. Integrar Cloud Firestore para almacenar los pedidos en la nube.
6. Implementar CRUD sobre los pedidos.
7. Actualizar el estado del pedido al cobrarlo y reflejar el cambio en Firestore.

8. Eliminar pedidos desde la aplicación y eliminar el documento correspondiente en Firestore.
9. Comprobar mediante pruebas y evidencias el funcionamiento de la aplicación.

4. Descripción general de la aplicación

La aplicación requiere autenticación antes de acceder al menú. El usuario puede consultar productos organizados por categorías, seleccionar una mesa, agregar o quitar productos y visualizar el total. Al realizar el pedido, la información se registra localmente y se almacena en Firestore. Desde VER PEDIDOS se pueden consultar pedidos pendientes, cobrarlos o eliminarlos.

5. Tecnologías utilizadas

5.1 Android Studio

¿Qué es?

Android Studio es el entorno de desarrollo oficial para crear aplicaciones para dispositivos Android.

¿Cómo se utilizó?

Se utilizó para desarrollar, organizar, compilar y ejecutar la aplicación MenuRestaurante3, además de administrar sus archivos Kotlin, XML, dependencias y configuración del proyecto.

5.2 Kotlin

¿Qué es?

Kotlin es un lenguaje de programación moderno utilizado principalmente para el desarrollo de aplicaciones Android.

¿Cómo se utilizó?

Se utilizó como lenguaje principal para desarrollar la lógica de la aplicación, incluyendo las actividades, modelos, controladores, clases DAO y operaciones relacionadas con los pedidos.

5.3 XML

¿Qué es?

XML es un lenguaje utilizado en Android para definir la estructura visual de las interfaces.

¿Cómo se utilizó?

Se utilizó para crear las pantallas de la aplicación, como el inicio de sesión, registro, menú principal y sección de pedidos.

5.4 RecyclerView

¿Qué es?

RecyclerView es un componente de Android que permite mostrar listas de elementos de manera eficiente.

¿Cómo se utilizó?

Se utilizó para mostrar dinámicamente los productos del menú y los pedidos realizados.

5.5 SQLite

¿Qué es?

SQLite es un sistema de base de datos relacional que permite almacenar información directamente en el dispositivo.

¿Cómo se utilizó?

Se utilizó como almacenamiento local para manejar información relacionada con productos, mesas y pedidos de la aplicación.

5.6 Firebase Authentication

¿Qué es?

Firebase Authentication es un servicio de Firebase que permite implementar sistemas de autenticación de usuarios.

¿Cómo se utilizó?

Se utilizó para implementar el registro, inicio de sesión y cierre de sesión mediante correo electrónico y contraseña.

5.7 Cloud Firestore

¿Qué es?

Cloud Firestore es una base de datos NoSQL en la nube proporcionada por Firebase, que permite almacenar información mediante colecciones y documentos.

¿Cómo se utilizó?

Se utilizó para almacenar los pedidos de la aplicación en la nube. Cada pedido se almacena como un documento dentro de la colección pedidos, incluyendo información de la mesa, fecha, estado, total y productos.

5.8 Arquitectura MVC

¿Qué es?

MVC (Modelo-Vista-Controlador) es un patrón de arquitectura que permite separar los datos, la interfaz y la lógica de una aplicación.

¿Cómo se utilizó?

Se utilizó para organizar el proyecto en modelos, vistas y controladores, complementándolo con clases DAO para separar el acceso a las bases de datos.

5.9 DAO (Data Access Object)

¿Qué es?

DAO (Data Access Object) es un patrón utilizado para separar las operaciones de acceso a datos de la lógica principal de una aplicación.

¿Cómo se utilizó?

Se utilizaron clases como ProductoDAO, PedidoDAO, PedidoListadoDAO y FirestoreDAO para realizar operaciones de lectura, almacenamiento, actualización y eliminación de información.

6. Arquitectura de la solución

La solución utiliza MVC para separar responsabilidades. La Vista contiene las interfaces y actividades; el Controlador gestiona la lógica de la aplicación; el Modelo representa los datos. Las clases DAO separan el acceso a SQLite y Firestore.

6.1 Modelo (Model)

Incluye las clases Producto, Pedido, PedidoItem y MenuItem, que representan los datos manejados por la aplicación.

6.2 Vista (View)

Está formada por las interfaces XML y las pantallas de Login, Registro, Menú principal y Pedidos.

6.3 Controlador (Controller)

Incluye PedidoController y la lógica de las Activities, encargadas de responder a las acciones del usuario, administrar pedidos y coordinar las operaciones.

6.4 Acceso a datos (DAO)

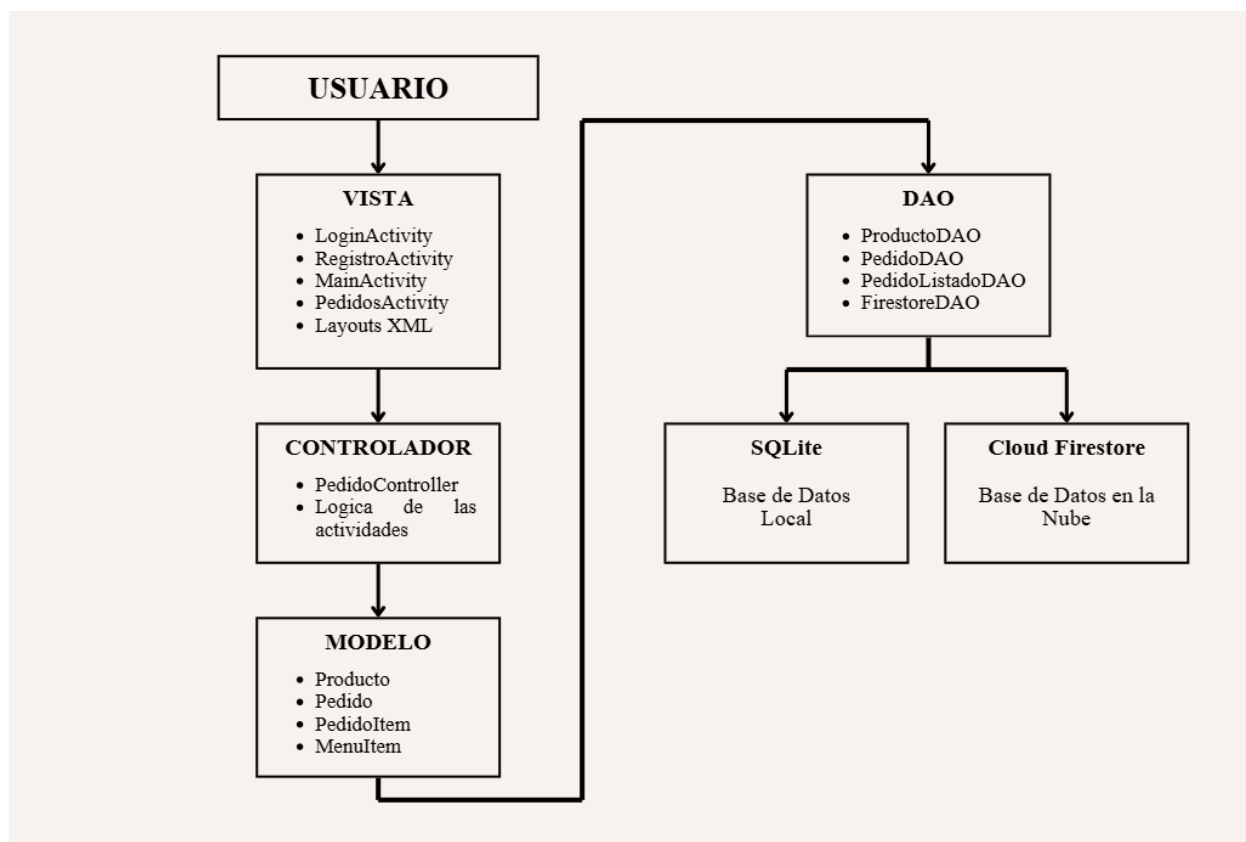
ProductoDAO, PedidoDAO y PedidoListadoDAO administran operaciones SQLite.

FirestoreDAO administra las operaciones de Cloud Firestore.

7. Diagrama de arquitectura MVC

Figura 1

Estructura de la arquitectura MVC de la App



Nota. El flujo representa la solución implementada: el usuario interactúa con la Vista, la lógica procesa la acción, los Modelos representan la información y los DAO realizan el acceso a los datos.

8. Integración con Firebase

8.1 Firebase Authentication

Firebase Authentication administra el registro e inicio de sesión mediante correo electrónico y contraseña, además del cierre de sesión desde la aplicación.

8.2 Cloud Firestore

Cloud Firestore almacena los pedidos en la colección pedidos. Cada documento contiene la información principal del pedido y su lista de productos. Al cobrar se actualiza el estado y al eliminar se elimina el documento.

9. Modelo de datos en Cloud Firestore

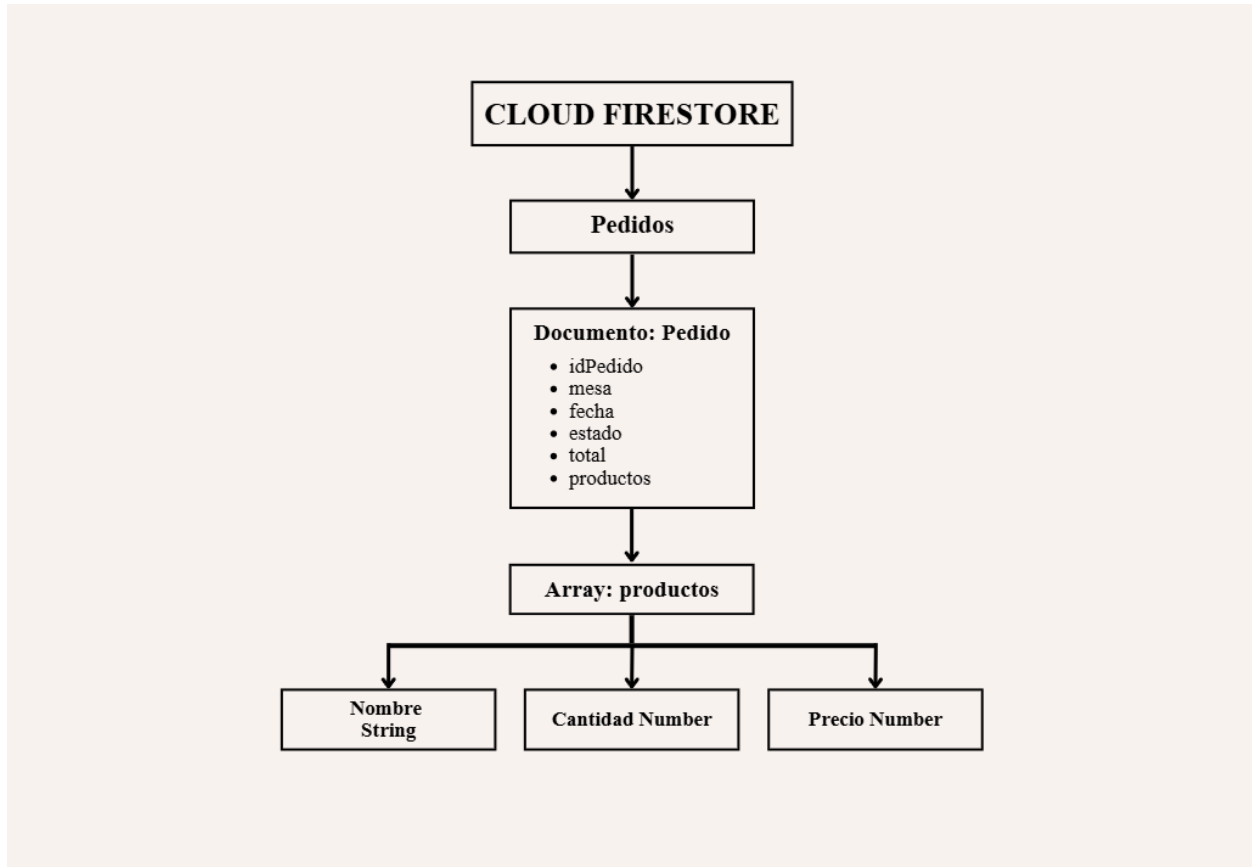
La colección principal utilizada es pedidos. Cada documento representa un pedido individual.

- idPedido: Identificador del pedido.
- mesa: Número de mesa.
- fecha: Fecha o marca de tiempo del pedido.
- estado: Estado del pedido, como Pendiente o Cobrado.
- total: Importe total.
- productos: Arreglo con los productos del pedido.
- Cada producto contiene nombre, cantidad y precio.

10. Diagrama del modelo de datos en Firestore

Figura 2

Modelo de datos de los pedidos en Firestore



11. Operaciones CRUD

11.1 Create – Crear

Al agregar productos, seleccionar una mesa y presionar REALIZAR PEDIDO, se crea el pedido y se registra en la base local y en Cloud Firestore.

11.2 Read – Consultar

La sección VER PEDIDOS consulta los pedidos pendientes almacenados localmente en SQLite y los presenta al usuario. Los pedidos creados también se almacenan en Cloud Firestore como respaldo y sincronización de la información en la nube.

11.3 Update – Actualizar

Al cobrar un pedido, su estado local cambia a Cobrado y el estado correspondiente en Firestore se actualiza a Cobrado.

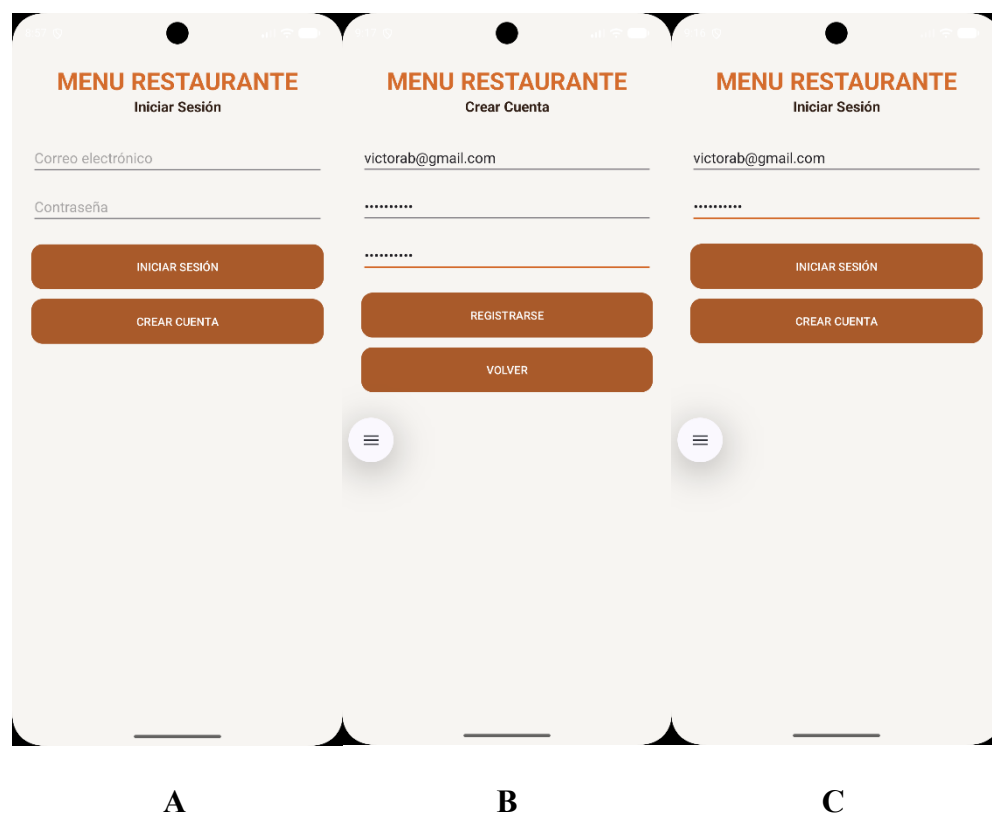
11.4 Delete – Eliminar

Al eliminar un pedido desde la aplicación, se elimina el registro local y también el documento correspondiente de la colección pedidos en Firestore.

12. Evidencias del funcionamiento

Figura 3

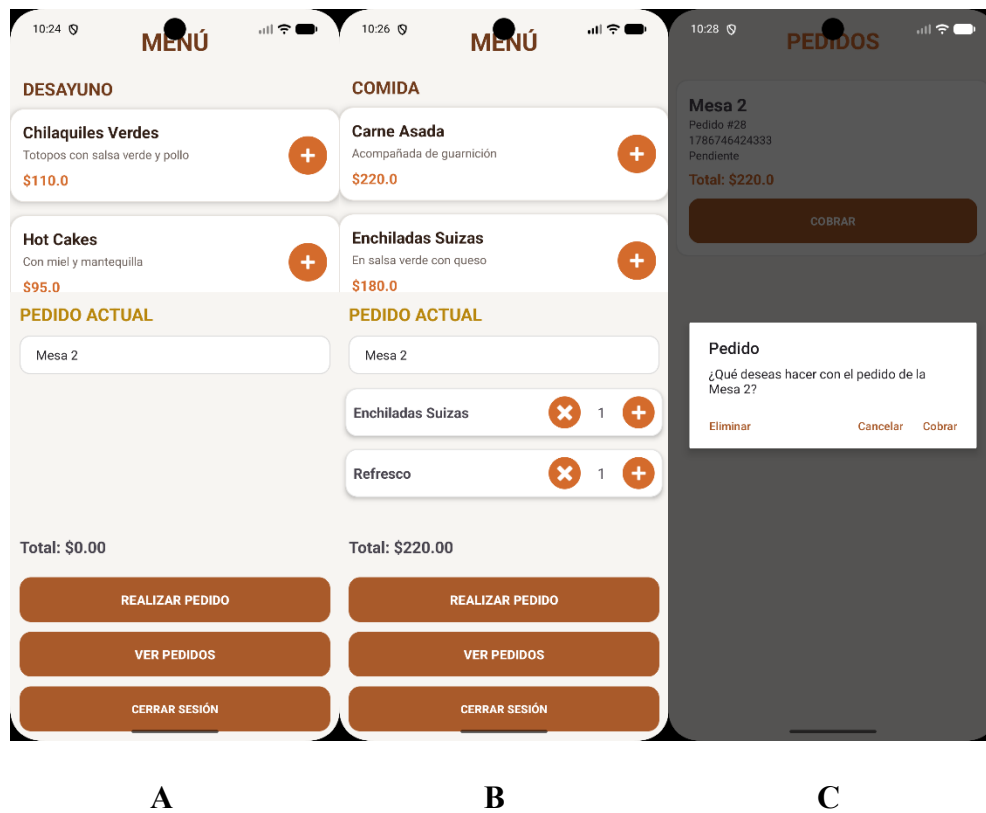
Pantalla del flujo de autenticación de usuarios



Nota. A) Pantalla de inicio de sesión, B) Pantalla de registro de usuario C) Inicio de sesión con datos ingresados.

Figura 4

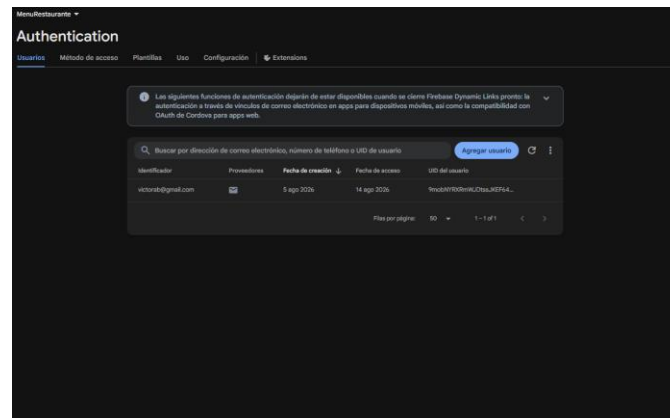
Interfaz para la selección de platillos y gestión de pedidos



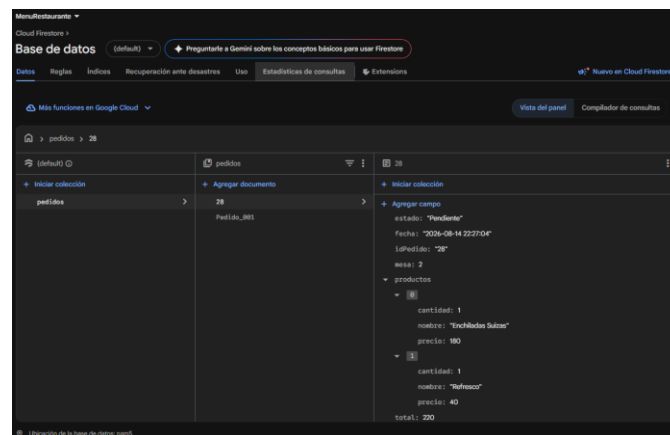
Nota. A) Vista del catálogo del menú por categorías, B) Agregación de productos al pedido actual de la mesa, C) Ventana de confirmación para la administración o cobro del pedido.

Figura 5

Paneles de control y gestión de datos en la consola web de Firebase



A



B

Nota. A) Registro de usuarios autenticados en el módulo de Firebase Authentication, B) Estructura de documentos y campos de la colección de pedidos en Cloud Firestore.

13. Pruebas realizadas

Tabla 1

Resultado de las pruebas realizadas

Prueba	Acción	Resultado
Registro	Registrar usuario	Correcto
Inicio de sesión	Ingresar con credenciales	Correcto
Cierre de sesión	Cerrar sesión	Correcto
Create	Realizar pedido	Correcto
Read	Consultar pedidos	Correcto
Update	Cobrar pedido	Correcto
Sincronización	Verificar cambio en Firestore	Correcto
Delete	Eliminar pedido	Correcto
Eliminación en nube	Verificar desaparición del documento	Correcto

14. Instalación y uso

14.1 Requisitos

- Android Studio.
- SDK de Android configurado para el proyecto.
- Proyecto Firebase configurado.
- Archivo google-services.json correspondiente al proyecto.

14.2 Instalación

1. Abrir MenuRestaurante3 en Android Studio.
2. Sincronizar Gradle.
3. Verificar la configuración de Firebase y google-services.json.
4. Comprobar Authentication y Cloud Firestore.
5. Ejecutar la aplicación en un emulador o dispositivo Android.

14.3 Uso

1. Registrar usuario.
2. Iniciar sesión.
3. Seleccionar mesa.
4. Agregar productos.
5. Realizar pedido.
6. Consultar VER PEDIDOS.
7. Cobrar o eliminar pedidos.
8. Cerrar sesión.

15. Conclusiones

En conclusión, el desarrollo de MenuRestaurante3 permitió implementar una aplicación móvil funcional para la gestión de pedidos de un restaurante utilizando Kotlin y Android Studio. A través de la aplicación es posible consultar el menú, seleccionar mesas, agregar productos a los pedidos, calcular el total y administrar los pedidos realizados.

La utilización de la arquitectura MVC, complementada con clases DAO, permitió organizar el proyecto de manera estructurada, separando la interfaz, los modelos, la lógica de la aplicación y el acceso a los datos. Esta organización facilitó el mantenimiento del código y permitió identificar con mayor claridad la función de cada componente dentro de la aplicación.

La integración con Firebase Authentication permitió implementar correctamente el registro de usuarios, inicio de sesión y cierre de sesión mediante correo electrónico y contraseña. Por otra parte, la integración con Cloud Firestore permitió almacenar los pedidos en la nube y comprobar que las operaciones realizadas desde la aplicación pueden reflejarse en la base de datos.

Durante las pruebas se comprobó el funcionamiento de las operaciones CRUD sobre los pedidos. La creación permite registrar nuevos pedidos, la consulta permite visualizar los pedidos pendientes, la actualización cambia el estado de un pedido cuando es cobrado y la eliminación permite borrar un pedido tanto de la aplicación como de Firestore.

Finalmente, el proyecto permitió integrar diferentes tecnologías de desarrollo móvil, almacenamiento local, autenticación y servicios en la nube en una sola aplicación. Los resultados obtenidos muestran que MenuRestaurante3 cumple con las funcionalidades planteadas y demuestra la integración entre una aplicación Android y una base de datos en la nube mediante Firebase.